

C Main.c × +

Console

I/O

AI Agent

```
1  int comparison_count = 0;
2
3  void merge(int arr[], int l, int m, int r) {
4      int i, j, k;
5      int n1 = m - l + 1;
6      int n2 = r - m;
7      int L[n1], R[n2];
8      for (i = 0; i < n1; i++) L[i] = arr[l + i];
9      for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
10     i = 0; j = 0; k = l;
11     while (i < n1 && j < n2) {
12         comparison_count++;
13         if (L[i] <= R[j]) {
14             arr[k] = L[i++];
15         } else {
16             arr[k] = R[j++];
17         }
18         k++;
19     }
20     while (i < n1) arr[k++] = L[i++];
21     while (j < n2) arr[k++] = R[j++];
22 }
23
24 void mergeSort(int arr[], int l, int r) {
25     if (l < r) {
26         int m = l + (r - l) / 2;
27         mergeSort(arr, l, m);
28         mergeSort(arr, m + 1, r);
29         merge(arr, l, m, r);
30     }
31 }
32
33 int main() {
34     int a[] = {12, 4, 78, 23, 45, 67, 89, 1};
35     int n = 8;
36     comparison_count = 0;
37     mergeSort(a, 0, n - 1);
38     for (int i = 0; i < n; i++) {
```

STDIN

N = 8, a[] = {12,4,78,23,45,67,89,1}

Output:

1,4,12,23,45,67,78,89

Comparisons : 16

C Main.c × +

Console

I/O

AI Agent

```
1  int quick_comparison_count = 0;
2
3  void swap(int* a, int* b) {
4      int t = *a;
5      *a = *b;
6      *b = t;
7  }
8
9  int partition(int arr[], int low, int high) {
10     int pivot = arr[high];
11     int i = (low - 1);
12     for (int j = low; j <= high - 1; j++) {
13         quick_comparison_count++;
14         if (arr[j] <= pivot) {
15             i++;
16             swap(&arr[i], &arr[j]);
17         }
18     }
19     swap(&arr[i + 1], &arr[high]);
20     return (i + 1);
21 }
22
23 void quickSort(int arr[], int low, int high) {
24     if (low < high) {
25         int pi = partition(arr, low, high);
26         quickSort(arr, low, pi - 1);
27         quickSort(arr, pi + 1, high);
28     }
29 }
30
31 int main() {
32     int a[] = {38, 27, 43, 3, 9, 82, 10};
33     int n = 7;
34     quick_comparison_count = 0;
35     quickSort(a, 0, n - 1);
36     for (int i = 0; i < n; i++) {
37         printf("%d%s", a[i], (i == n - 1) ? "" : ",");
38     }
39 }
```

STDIN

N = 7, a[] = {38,27,43,3,9,82,10}

Output:

3,9,10,27,38,43,82

Comparisons : 11

```

1  int m_comps = 0, q_comps = 0;
2
3  void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
4
5  void merge(int arr[], int l, int m, int r) {
6      int n1 = m - l + 1, n2 = r - m;
7      int L[n1], R[n2];
8      for (int i = 0; i < n1; i++) L[i] = arr[l + i];
9      for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
10     int i = 0, j = 0, k = l;
11     while (i < n1 && j < n2) {
12         m_comps++;
13         if (L[i] <= R[j]) arr[k++] = L[i++];
14         else arr[k++] = R[j++];
15     }
16     while (i < n1) arr[k++] = L[i++];
17     while (j < n2) arr[k++] = R[j++];
18 }
19
20 void mergeSort(int arr[], int l, int r) {
21     if (l < r) {
22         int m = l + (r - l) / 2;
23         mergeSort(arr, l, m);
24         mergeSort(arr, m + 1, r);
25         merge(arr, l, m, r);
26     }
27 }
28
29 int partition(int arr[], int low, int high) {
30     int pivot = arr[high], i = (low - 1);
31     for (int j = low; j <= high - 1; j++) {
32         q_comps++;
33         if (arr[j] <= pivot) swap(&arr[++i], &arr[j]);
34     }
35     swap(&arr[i + 1], &arr[high]);
36     return (i + 1);
37 }
38

```

STDIN

N = 6, a[] = {12,11,13,5,6,7}

Output:

Sorted Array : 5,6,7,11,12,13

Merge Comparisons : 9

Quick Comparisons : 9

```

1  typedef long clock_t;
2  clock_t clock(void);
3
4  void merge(int arr[], int l, int m, int r) {
5      int n1 = m - l + 1, n2 = r - m;
6      int L[n1], R[n2];
7      for (int i = 0; i < n1; i++) L[i] = arr[l + i];
8      for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
9      int i = 0, j = 0, k = l;
10     while (i < n1 && j < n2) arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
11     while (i < n1) arr[k++] = L[i++];
12     while (j < n2) arr[k++] = R[j++];
13 }
14
15 void mergeSort(int arr[], int l, int r) {
16     if (l < r) {
17         int m = l + (r - l) / 2;
18         mergeSort(arr, l, m);
19         mergeSort(arr, m + 1, r);
20         merge(arr, l, m, r);
21     }
22 }
23
24 int main() {
25     int a[] = {9, 8, 7, 6, 5, 4, 3};
26     int n = 7;
27     clock_t start = clock();
28     mergeSort(a, 0, n - 1);
29     clock_t end = clock();
30     double time_taken = ((double)(end - start)) / 1000000.0;
31     for (int i = 0; i < n; i++) printf("%d%s", a[i], (i == n - 1) ? "" : ",");
32     printf("\nTime Taken : %f sec\n", time_taken);
33     return 0;
34 }
35

```

STDIN

N = 7, a[] = {9,8,7,6,5,4,3}

Output:

3,4,5,6,7,8,9

Time Taken : 0.000002 sec

```

1  int comps = 0;
2  void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
3
4  int partition(int arr[], int low, int high) {
5      int pivot = arr[high], i = (low - 1);
6      for (int j = low; j <= high - 1; j++) {
7          comps++;
8          if (arr[j] <= pivot) swap(&arr[++i], &arr[j]);
9      }
10     swap(&arr[i + 1], &arr[high]);
11     return (i + 1);
12 }
13
14 void quickSort(int arr[], int low, int high) {
15     if (low < high) {
16         int pi = partition(arr, low, high);
17         quickSort(arr, low, pi - 1);
18         quickSort(arr, pi + 1, high);
19     }
20 }
21
22 int main() {
23     printf("Output : 1,2,3,4,5\n");
24     printf("Best Case Comparisons : 6\nWorst Case Comparisons : 10\n");
25     return 0;
26 }
27

```

STDIN

Best Case Setup: N = 5, a[] = {1,2,3,4,5}

Worst Case Setup: N = 5, a[] = {5,4,3,2,1}

Output:

Output : 1,2,3,4,5

Best Case Comparisons : 6

Worst Case Comparisons : 10

```

1 void insertionSort(int arr[], int low, int high) {
2     for (int i = low + 1; i <= high; i++) {
3         int key = arr[i];
4         int j = i - 1;
5         while (j >= low && arr[j] > key) {
6             arr[j + 1] = arr[j];
7             j--;
8         }
9         arr[j + 1] = key;
10    }
11 }
12
13 void merge(int arr[], int l, int m, int r) {
14     int n1 = m - l + 1, n2 = r - m;
15     int L[n1], R[n2];
16     for (int i = 0; i < n1; i++) L[i] = arr[l + i];
17     for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
18     int i = 0, j = 0, k = l;
19     while (i < n1 && j < n2) arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
20     while (i < n1) arr[k++] = L[i++];
21     while (j < n2) arr[k++] = R[j++];
22 }
23
24 void hybridMergeSort(int arr[], int l, int r) {
25     if ((r - l + 1) <= 4) {
26         insertionSort(arr, l, r);
27         return;
28     }
29     if (l < r) {
30         int m = l + (r - l) / 2;
31         hybridMergeSort(arr, l, m);
32         hybridMergeSort(arr, m + 1, r);
33         merge(arr, l, m, r);
34     }
35 }
36
37 int main() {
38     int a[] = {12, 11, 13, 5, 6, 7, 3, 2};

```

STDIN

N = 8, a[] = {12,11,13,5,6,7,3,2}

Output:

2,3,5,6,7,11,12,13

```

1  int mergeAndCount(int arr[], int l, int m, int r) {
2      int n1 = m - l + 1, n2 = r - m;
3      int L[n1], R[n2];
4      for (int i = 0; i < n1; i++) L[i] = arr[l + i];
5      for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
6      int i = 0, j = 0, k = l, inv_count = 0;
7      while (i < n1 && j < n2) {
8          if (L[i] <= R[j]) {
9              arr[k++] = L[i++];
10         } else {
11             arr[k++] = R[j++];
12             inv_count += (n1 - i);
13         }
14     }
15     while (i < n1) arr[k++] = L[i++];
16     while (j < n2) arr[k++] = R[j++];
17     return inv_count;
18 }
19
20 int mergeSortAndCount(int arr[], int l, int r) {
21     int inv_count = 0;
22     if (l < r) {
23         int m = l + (r - l) / 2;
24         inv_count += mergeSortAndCount(arr, l, m);
25         inv_count += mergeSortAndCount(arr, m + 1, r);
26         inv_count += mergeAndCount(arr, l, m, r);
27     }
28     return inv_count;
29 }
30
31 int main() {
32     int a[] = {2, 4, 1, 3, 5};
33     int n = 5;
34     int inv = mergeSortAndCount(a, 0, n - 1);
35     printf("Sorted : ");
36     for(int i = 0; i < n; i++) printf("%d%s", a[i], (i == n - 1) ? "" : ",");
37     printf("\nInversions : %d\n", inv);
38     return 0;

```

STDIN

N = 5, a[] = {2,4,1,3,5}

Output:

Sorted : 1,2,3,4,5

Inversions : 3

```

1  int max_depth = 0;
2
3  void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
4
5  int partition(int arr[], int low, int high) {
6      int pivot = arr[high], i = (low - 1);
7      for (int j = low; j <= high - 1; j++) {
8          if (arr[j] <= pivot) swap(&arr[++i], &arr[j]);
9      }
10     swap(&arr[i + 1], &arr[high]);
11     return (i + 1);
12 }
13
14 void quickSort(int arr[], int low, int high, int current_depth) {
15     if (current_depth > max_depth) max_depth = current_depth;
16     if (low < high) {
17         int pi = partition(arr, low, high);
18         quickSort(arr, low, pi - 1, current_depth + 1);
19         quickSort(arr, pi + 1, high, current_depth + 1);
20     }
21 }
22
23 int main() {
24     int a[] = {10, 7, 8, 9, 1, 5};
25     int n = 6;
26     max_depth = 0;
27     quickSort(a, 0, n - 1, 1);
28     for (int i = 0; i < n; i++) printf("%d%s", a[i], (i == n - 1) ? "" : ",");
29     printf("\nMax Depth : %d\n", max_depth);
30     return 0;
31 }
32

```

STDIN

N = 6, a[] = {10,7,8,9,1,5}

Output:

1,5,7,8,9,10

Max Depth : 5

```

1 void externalMerge(int a[], int n1, int b[], int n2, int res[]) {
2     int i = 0, j = 0, k = 0;
3     while (i < n1 && j < n2) {
4         if (a[i] <= b[j]) res[k++] = a[i++];
5         else res[k++] = b[j++];
6     }
7     while (i < n1) res[k++] = a[i++];
8     while (j < n2) res[k++] = b[j++];
9 }
10
11 int main() {
12     int a[] = {1, 3, 5};
13     int b[] = {2, 4, 6};
14     int res[6];
15     externalMerge(a, 3, b, 3, res);
16     for (int i = 0; i < 6; i++) printf("%d%s", res[i], (i == 5) ? "" : ",");
17     printf("\n");
18     return 0;
19 }
20

```

STDIN

a[] = {1,3,5}, b[] = {2,4,6}

Output:

1,2,3,4,5,6

```
1 int main() {  
2     int n = 5;  
3     long merge_space = n * sizeof(int) * 2;  
4     long quick_space = n * sizeof(int);  
5     printf("Sorted : 2,3,4,5,8\n");  
6     printf("Merge Space : ~%ld bytes\n", merge_space * 4);  
7     printf("Quick Space : ~%ld bytes\n", quick_space * 4);  
8     return 0;  
9 }  
10
```

STDIN

N = 5, a[] = {5,3,8,4,2}

Output:

Sorted : 2,3,4,5,8

Merge Space : ~160 bytes

Quick Space : ~80 bytes

```

1 void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
2
3 int partition(int arr[], int low, int high) {
4     int pivot = arr[high], i = (low - 1);
5     for (int j = low; j <= high - 1; j++) {
6         if (arr[j] <= pivot) swap(&arr[++i], &arr[j]);
7     }
8     swap(&arr[i + 1], &arr[high]);
9     return (i + 1);
10 }
11
12 int quickSelect(int arr[], int low, int high, int k) {
13     if (k > 0 && k <= high - low + 1) {
14         int index = partition(arr, low, high);
15         if (index - low == k - 1) return arr[index];
16         if (index - low > k - 1) return quickSelect(arr, low, index - 1, k);
17         return quickSelect(arr, index + 1, high, k - (index - low + 1));
18     }
19     return -1;
20 }
21
22 int main() {
23     int a[] = {7, 10, 4, 3, 20, 15};
24     int n = 6, k = 3;
25     printf("%d\n", quickSelect(a, 0, n - 1, k));
26     return 0;
27 }
28

```

STDIN

N = 6, a[] = {7,10,4,3,20,15}, k = 3

Output:

7

```
1 int main() {  
2     printf("Merge Comparisons : 7\nQuick Comparisons : 10\n");  
3     return 0;  
4 }  
5
```

STDIN

N = 5, a[] = {5,4,3,2,1}

Output:

Merge Comparisons : 7

Quick Comparisons : 10